List of HTTP status codes

This article lists standard and notable non-standard HTTP response status codes. Standardized codes are defined by IETF as documented in Request for Comments (RFC) publications and maintained by the IANA.[1] Other, non-standard values are used by various servers. The descriptive text after the numeric code – the *reason phrase* – is shown here with typical value, but in practice, can be different or omitted.

Standard codes

Status codes defined by IETF are listed below. Emphasized terms *must*, *must not* and *should* are interpretation guidelines as given by RFC 2119 (https://www.rfc-editor.org/rfc/rfc2119).

1xx informational response

An informational response indicates that the request was received and understood and is being processed. It alerts the client to wait for a final response. The message does not contain a body. As the HTTP/1.0 standard did not define any 1xx status codes, servers *must not* send a 1xx response to an HTTP/1.0 compliant client except under experimental conditions.

100 Continue
: The server has received the request headers and the client should proceed to send the request body (in the case of a request for which a body needs to be sent, such as a POST request). Sending a large request body to a server after a request has been rejected for inappropriate headers would be inefficient. To have a server check the request's headers, a client must send `Expect: 100-continue` as a header in its initial request and receive a `100 Continue` status code in response before sending the body. If the client receives an error code such as 403 (Forbidden) or 405 (Method Not Allowed) then it should not send the request's body. The response `417 Expectation Failed` indicates that the request should be repeated without the `Expect` header as it indicates that the server does not support expectations (this is the case, for example, of HTTP/1.0 servers).[2]:§10.1.1

101 Switching Protocols
: The requester has asked the server to switch protocols and the server has agreed to do so.

102 Processing (WebDAV; RFC 2518)
: A WebDAV request may contain many sub-requests involving file operations, requiring a long time to complete the request. This code indicates that the server has received and is processing the request, but no response is available yet.[3] This prevents the client from timing out and assuming the request was lost. The status code is deprecated.[4]

103 Early Hints (RFC 8297)
: Used to return some response headers before final HTTP message.[5]

2xx success

A success status indicates that the action requested by the client was received, understood, and accepted.[1]

200 OK
Standard response for successful HTTP requests. The actual response will depend on the request method used. In a GET request, the response will contain an entity corresponding to the requested resource. In a POST request, the response will contain an entity describing or containing the result of the action.

201 Created
The request has been fulfilled, resulting in the creation of a new resource.[6]

202 Accepted
The request has been accepted for processing, but the processing has not been completed. The request might or might not be eventually acted upon, and may be disallowed when processing occurs.

203 Non-Authoritative Information (since HTTP/1.1)
The server is a transforming proxy (e.g. a *Web accelerator*) that received a 200 OK from its origin, but is returning a modified version of the origin's response.[2]:§15.3.4[2]:§7.7

204 No Content
The server successfully processed the request, and is not returning any content.

205 Reset Content
The server successfully processed the request, asks that the requester reset its document view, and is not returning any content.

206 Partial Content
The server is delivering only part of the resource (byte serving) due to a range header sent by the client. The range header is used by HTTP clients to enable resuming of interrupted downloads, or split a download into multiple simultaneous streams.

207 Multi-Status (WebDAV; RFC 4918)
The message body that follows is by default an XML message and can contain a number of separate response codes, depending on how many sub-requests were made.[7]

208 Already Reported (WebDAV; RFC 5842)
The members of a DAV binding have already been enumerated in a preceding part of the (multistatus) response, and are not being included again.

226 IM Used (RFC 3229)
The server has fulfilled a request for the resource, and the response is a representation of the result of one or more instance-manipulations applied to the current instance.[8]

3xx redirection

A 3xx status indicates that the client must take additional action, generally URL redirection, to complete the request.[1] A user agent may carry out the additional action with no user interaction if the method used in the additional request is GET or HEAD. A user agent should prevent cyclical redirects.[2]:§15.4

300 Multiple Choices
Indicates multiple options for the resource from which the client may choose (via agent-driven content negotiation). For example, this code could be used to present multiple video format options, to list files with different filename extensions, or to suggest word-sense disambiguation.

301 Moved Permanently
The link target was moved such that the request and future similar requests should be redirected to the given URI. If a client has link-editing capabilities, it should update references to the request URL. The response is cacheable unless indicated otherwise.

Except for a GET request, the body should contain a hyperlink to the new URL(s). Except for a GET or HEAD request, the client must ask the user before redirecting.[9]

This code is considered best practice for upgrading users from HTTP to HTTPS. Both Bing and Google recommend using this code to change the URL of a page as it is shown in search engine results, providing that URL will permanently change and is not due to be changed again any time soon.[10][11]

302 Found

Indicates that the resource is accessible via an alternate URL indicated in the Location header field. The HTTP/1.0 specification (which used reason phrase "Moved Temporarily") required the client to redirect with the same method,[12] but popular browsers instead changed the request to GET.[13] For this reason, HTTP/1.1 (RFC 2616 (https://www.rfc-editor.org/rfc/rfc2616)) added two status codes: 303 which requires changing the request to a GET and 307 which preserves the original request type. Despite the greater clarity provided by this disambiguation, the 302 code is still used in web frameworks to preserve compatibility with browsers that do not support HTTP/1.1.[14][2]:§15.4 As a consequence, RFC 7231 (https://www.rfc-editor.org/rfc/rfc7231) (the update of RFC 2616 (https://www.rfc-editor.org/rfc/rfc2616)) changes the definition to allow user agents to rewrite POST to GET.[15]

303 See Other (since HTTP/1.1)

If a server responds to a POST or other non-idempotent request with this code and a location header field, the client is expected to issue a GET request to the specified location. To trigger a request to the target resource using the same method, the server responds with 307 instead.

Use of this code has been proposed[16] as one way of responding to a request for a URI that identifies a real-world object according to Semantic Web theory (the other being the use of *hash URIs*).[17][16] For example, if `http://www.example.com/id/alice` identifies a person, Alice, then it would be inappropriate for a server to respond to a GET request with 200 OK, as the server could not deliver Alice herself. Instead, the server would respond with 303 to redirect to a URI that provides a description of the person Alice.[16]

Sometimes, this code is used when providing an HTTP-based web API that needs to respond to the caller immediately, but continue executing asynchronously, such as a long-lived image conversion. The web API provides a status check URI that allows the client to check on the operation's status. When complete, the response may contain this status code and a redirect URI to the final result.[18]

304 Not Modified

Indicates that the resource has not been modified since the version specified by the request headers If-Modified-Since or If-None-Match. In such case, there is no need to retransmit the resource since the client still has a previously-downloaded copy.

305 Use Proxy (since HTTP/1.1)

The requested resource is available only through a proxy, the address for which is provided in the response. For security reasons, many HTTP clients (such as Mozilla Firefox and Internet Explorer) do not obey this status code.[19]

306 Switch Proxy

No longer used. Originally meant "Subsequent requests should use the specified proxy."

307 Temporary Redirect (since HTTP/1.1)

In this case, the request should be repeated with another URI; however, future requests should still use the original URI. In contrast to how 302 was historically implemented, the request method is not allowed to be changed when reissuing the original request. For example, a POST request should be repeated using another POST request.

308 Permanent Redirect

This and all future requests should be directed to the given URI. 308 parallels the behavior of 301, but *does not allow the HTTP method to change*. So, for example,

submitting a form to a permanently redirected resource may continue smoothly.

4xx client error

A 4xx status code is for situations in which an error seems to have been caused by the client. Except when responding to a HEAD request, the server *should* include an entity containing an explanation of the error situation, and whether it is a temporary or permanent condition. These status codes are applicable to any request method. User agents *should* display any included entity to the user.

404 error on Wikimedia

400 Bad Request
The server cannot or will not process the request due to an apparent client error (e.g., malformed request syntax, size too large, invalid request message framing, or deceptive request routing).

401 Unauthorized
Similar to 403 Forbidden, but specifically for use when authentication is required and has failed or has not yet been provided. The response must include a WWW-Authenticate header field containing a challenge applicable to the requested resource. See Basic access authentication and Digest access authentication. 401 semantically means "unauthenticated", the user does not have valid authentication credentials for the target resource.

402 Payment Required
Reserved for future use. The original intention was that this code might be used as part of some form of digital cash or micropayment scheme, as proposed, for example, by GNU Taler,[20] but that has not yet happened, and this code is not widely used. Google Developers API uses this status if a particular developer has exceeded the daily limit on requests.[21] Sipgate uses this code if an account does not have sufficient funds to start a call.[22] Shopify uses this code when the store has not paid their fees and is temporarily disabled.[23] Stripe uses this code for failed payments where parameters were correct, for example blocked fraudulent payments.[24] Cloudflare Turnstile uses this code when requesting resources with cURL.

403 Forbidden
The request was valid, but the server refuses action. This may be due to the user not having permission to a resource or needing an account of some sort, or attempting a prohibited action (e.g. creating a duplicate record where only one is allowed). This code is also typically used if the request provided authentication by answering the WWW-Authenticate header field challenge, but the server did not accept that authentication. The request should not be repeated.
This code differs from 401 in that while 401 is returned when the client has not authenticated, and implies that a successful response may be returned following valid authentication, 403 is returned when the client is not permitted access to the resource despite providing authentication such as insufficient permissions of the authenticated account.
The Apache web server returns 403 in response to a request for URL[25] paths that corresponded to a file system directory when directory listing is disabled and there is no Directory Index directive to specify an existing file to be returned to the browser. Some administrators configure the Mod proxy extension to block such requests and this will also return 403. IIS responds in the same way when directory listings are denied in that server. In WebDAV, 403 is returned if the client issued a PROPFIND request but did not also issue the required Depth header or issued a Depth header of infinity.[25]

404 Not Found

The requested resource could not be found but may be available in the future. Subsequent requests by the client are permissible.

405 Method Not Allowed
A request method is not supported for the requested resource (for example, a GET request on a form that requires data to be presented via POST, or a PUT request on a read-only resource).

406 Not Acceptable
The requested resource is capable of generating only content not acceptable according to the Accept headers sent in the request. See Content negotiation.

407 Proxy Authentication Required
The client must first authenticate itself with the proxy.

408 Request Timeout
The server timed out waiting for the request. According to HTTP specifications: "The client did not produce a request within the time that the server was prepared to wait. The client MAY repeat the request without modifications at any later time."

409 Conflict
Indicates that the request could not be processed because of conflict in the current state of the resource, such as an edit conflict between multiple simultaneous updates.[26]

410 Gone
Indicates that the resource requested was previously in use but is no longer available and will not be available again. This should be used when a resource has been intentionally removed and the resource should be purged. Upon receiving a 410 status code, the client should not request the resource in the future. Clients such as search engines should remove the resource from their indices. Most use cases do not require clients and search engines to purge the resource, and a "404 Not Found" may be used instead.

411 Length Required
The request did not specify the length of its content, which is required by the requested resource.

412 Precondition Failed
The server does not meet one of the preconditions that the requester put on the request header fields.

413 Content Too Large
The request is larger than the server is willing or able to process. Previously called "Request Entity Too Large" and "Payload Too Large".[27]:§10.4.14[2]:§15.5.14

414 URI Too Long
The URI provided was too long for the server to process. Often the result of too much data being encoded as a query-string of a GET request, in which case it should be converted to a POST request. Called "Request-URI Too Long" previously.[27]:§10.4.15

415 Unsupported Media Type
The request entity has a media type which the server or resource does not support. For example, the client uploads an image as image/svg+xml, but the server requires that images use a different format.

416 Range Not Satisfiable
The client has asked for a portion of the file (byte serving), but the server cannot supply that portion. For example, if the client asked for a part of the file that lies beyond the end of the file. Called "Requested Range Not Satisfiable" previously.[27]:§10.4.17

417 Expectation Failed
The server cannot meet the requirements of the Expect request-header field.[28]

418 I'm a teapot (RFC 2324, RFC 7168)
This code was defined in 1998 as one of the traditional IETF April Fools' jokes, in RFC 2324, *Hyper Text Coffee Pot Control Protocol*, and is not expected to be implemented by actual HTTP servers. The RFC specifies this code should be returned by teapots requested to brew coffee.[29] This HTTP status is used as an Easter egg in some websites, such as Google.com's "I'm a teapot" easter egg.[30][31] Sometimes, this status

code is also used as a response to a blocked request, instead of the more appropriate 403 Forbidden.[32][33]

421 Misdirected Request
The request was directed at a server that is not able to produce a response (for example because of connection reuse).

422 Unprocessable Content
The request was well-formed (i.e., syntactically correct) but could not be processed.[2]:§15.5.21

423 Locked (WebDAV; RFC 4918)
The resource that is being accessed is locked.[7]

424 Failed Dependency (WebDAV; RFC 4918)
The request failed because it depended on another request and that request failed (e.g., a PROPPATCH).[7]

425 Too Early (RFC 8470)
Indicates that the server is unwilling to risk processing a request that might be replayed.

426 Upgrade Required
The client should switch to a different protocol such as TLS/1.3, given in the Upgrade header field.

428 Precondition Required (RFC 6585)
The origin server requires the request to be conditional. Intended to prevent the 'lost update' problem, where a client GETs a resource's state, modifies it, and PUTs it back to the server, when meanwhile a third party has modified the state on the server, leading to a conflict.[34]

429 Too Many Requests (RFC 6585)
The user has sent too many requests in a given amount of time. Intended for use with rate-limiting schemes.[34]

431 Request Header Fields Too Large (RFC 6585)
The server is unwilling to process the request because either an individual header field, or all the header fields collectively, are too large.[34]

451 Unavailable For Legal Reasons (RFC 7725)
A server operator has received a legal demand to deny access to a resource or to a set of resources that includes the requested resource.[35] The code 451 was chosen as a reference to the novel *Fahrenheit 451*.[36]

5xx server error

5xx status indicates that the server is aware that it has encountered an error or is otherwise incapable of performing the request. Except when responding to a HEAD request, the server *should* include an entity containing an explanation of the error situation, and indicate whether it is a temporary or permanent condition. Likewise, user agents *should* display any included entity to the user. These response codes are applicable to any request method.

500 Internal Server Error
A generic error message, given when an unexpected condition was encountered and no more specific message is suitable.

501 Not Implemented
The server either does not recognize the request method, or it lacks the ability to fulfil the request. Usually this implies future availability (e.g., a new feature of a web-service API).

502 Bad Gateway
The server was acting as a gateway or proxy and received an invalid response from the upstream server.

503 Service Unavailable

The server cannot handle the request (because it is overloaded or down for maintenance). Generally, this is a temporary state.[37]

504 Gateway Timeout
The server was acting as a gateway or proxy and did not receive a timely response from the upstream server.

505 HTTP Version Not Supported
The server does not support the HTTP version used in the request.

506 Variant Also Negotiates (RFC 2295)
Transparent content negotiation for the request results in a circular reference.[38]

507 Insufficient Storage (WebDAV; RFC 4918)
The server is unable to store the representation needed to complete the request.[7]

508 Loop Detected (WebDAV; RFC 5842)
The server detected an infinite loop while processing the request (sent instead of 208 Already Reported).

510 Not Extended (RFC 2774)
Further extensions to the request are required for the server to fulfil it.[39]

511 Network Authentication Required (RFC 6585)
The client needs to authenticate to gain network access. Intended for use by intercepting proxies used to control access to the network (e.g., "captive portals" used to require agreement to Terms of Service before granting full Internet access via a Wi-Fi hotspot).[34]

Nonstandard codes

The following codes are used by various web servers but not specified by an IETF standard.

Internet Information Services

Microsoft's Internet Information Services (IIS) web server expands the 4xx error space to signal errors with the client's request. IIS sometimes uses additional decimal sub-codes for more specific information,[40] however these sub-codes only appear in the response payload and in documentation, not in the place of an actual HTTP status code.

440 Login Time-out
The client's session has expired and must log in again.[41]

449 Retry With
The server cannot honor the request because the user has not provided the required information.[42]

450 Blocked by Windows Parental Controls
Indicates that Windows Parental Controls block access to the requested webpage.[43]

451 Redirect
Used in Exchange ActiveSync when either a more efficient server is available or the server cannot access the users' mailbox.[44] The client is expected to re-run the HTTP AutoDiscover operation to find a more appropriate server.[45]

nginx

The nginx web server software expands the 4xx error space to signal issues with the client's request.[46][47]

444 No Response

Used internally[48] to instruct the server to return no information to the client and close the connection immediately.

494 Request header too large
Client sent too large request or too long header line.

495 SSL Certificate Error
An expansion of the 400 Bad Request response code, used when the client has provided an invalid client certificate.

496 SSL Certificate Required
An expansion of the 400 Bad Request response code, used when a client certificate is required but not provided.

497 HTTP Request Sent to HTTPS Port
An expansion of the 400 Bad Request response code, used when the client has made a HTTP request to a port listening for HTTPS requests.

499 Client Closed Request
Used when the client has closed the request before the server could send a response.

Cloudflare

Cloudflare's reverse proxy service expands the 5xx series of errors space to signal issues with the origin server.[49]

520 Web Server Returned an Unknown Error
The origin server returned an empty, unknown, or unexpected response to Cloudflare.[50]

521 Web Server Is Down
The origin server refused connections from Cloudflare. Security solutions at the origin may be blocking legitimate connections from certain Cloudflare IP addresses.[51]

522 Connection Timed Out
Cloudflare timed out contacting the origin server.[52]

523 Origin Is Unreachable
Cloudflare could not contact the origin server.[53]

524 A Timeout Occurred
Cloudflare was able to complete a TCP connection to the origin server, but the origin did not provide a timely HTTP response.[54]

525 SSL Handshake Failed
Cloudflare could not negotiate a SSL/TLS handshake with the origin server.[55][56]

526 Invalid SSL Certificate
Cloudflare could not validate the SSL certificate on the origin web server.[57][58] Also used by Cloud Foundry's gorouter.

527 Railgun Error (obsolete)
Error 527 indicated an interrupted connection between Cloudflare and the origin server's Railgun server.[59] This error is obsolete as Cloudflare has deprecated Railgun.

530 Origin Unavailable
Cloudflare was unable to resolve the origin hostname, preventing it from establishing a connection to the origin server. The body of the response contains an 1xxx error.[60][61]

AWS Elastic Load Balancing

Amazon Web Services' Elastic Load Balancing adds a few custom return codes to signal issues either with the client request or with the origin server.[62]

000

Returned with an HTTP/2 GOAWAY frame if the compressed length of any of the headers exceeds 8K bytes or if more than 10K requests are served through one connection.[62]

460

Client closed the connection with the load balancer before the idle timeout period elapsed. Typically, when client timeout is sooner than the Elastic Load Balancer's timeout.[62]

463

The load balancer received an X-Forwarded-For request header with more than 30 IP addresses.[62]

464

Incompatible protocol versions between Client and Origin server.[62]

561 Unauthorized

An error around authentication returned by a server registered with a load balancer. A listener rule is configured to authenticate users, but the identity provider (IdP) returned an error code when authenticating the user.[62]

Apache

Used by Apache HTTP Server.

509 Bandwidth Limit Exceeded

The server has exceeded the bandwidth specified by the server administrator; this is often used by shared hosting providers to limit the bandwidth of customers.[63] Also used by cPanel.

Laravel framework

Used by Laravel Framework.

419 Page Expired

A CSRF Token is missing or expired.[64]

Spring Framework

Used by Spring Framework.

420 Method Failure

A deprecated response status proposed during the development of WebDAV[65] used by the Spring Framework when a method has failed.[66]

Twitter

Used by Twitter.

420 Enhance Your Calm

Returned by version 1 of the Twitter Search and Trends API when the client is being rate limited; versions 1.1 and later use the 429 Too Many Requests response code instead.[67] The phrase "Enhance your calm" comes from the 1993 movie *Demolition Man*, and its association with this number is likely a reference to cannabis.

Shopify

Used by Shopify.

430 Request Header Fields Too Large
: A deprecated response used by Shopify, instead of the 429 Too Many Requests response code, when too many URLs are requested within a certain time frame.[68]

430 Shopify Security Rejection
: Used by Shopify to signal that the request was deemed malicious.[69]

530 Origin DNS Error
: Indicates that Cloudflare can't resolve the requested DNS record.[69]

540 Temporarily Disabled
: Indicates that the requested endpoint has been temporarily disabled.[69]

783 Unexpected Token
: Indicates that the request includes a JSON syntax error.[69]

ArcGIS Server

Used by ArcGIS Server.

498 Invalid Token
: Indicates an expired or otherwise invalid token.[70]

499 Token Required
: Indicates that a token is required but was not submitted.[70]

cPanel

Used by cPanel.

508 Resource Limit Is Reached
: Used instead of 503 when the server's account has exceeded the resources assigned to it, such as CPU/RAM usage or number of concurrent processes.[71]

SSLLabs server testing API

Used by Qualys in the SSLLabs server testing API.

529 Site is overloaded
: Signals that the site can not process the request.[72]

Pantheon Systems web platform

Used by the Pantheon Systems web platform.

530 Site is frozen
: Indicates a site that has been frozen due to inactivity.[73]

LinkedIn

Used by LinkedIn.

999 Request denied

Related to being blocked/walled or unable to access their webpages without first signing in.[74]

Miscellaneous

218 This is fine
: An informal catch-all error condition, widely attributed to the Apache HTTP server to allow for the passage of message bodies through the server when the `ProxyErrorOverride` setting is enabled, though the status code and behavior is not part of any official Apache specification. The association between this status code and Apache has been attributed to unsourced additions to Wikipedia, which were subsequently picked up by other reference material, creating circular references.[75]

598 Network read timeout error
: An informal convention used by some HTTP proxies to signal a network read timeout behind the proxy to a client in front of the proxy.[76]

599 Network Connect Timeout Error
: An error used by some HTTP proxies to signal a network connect timeout behind the proxy to a client in front of the proxy.

See also

- Custom error pages
- List of FTP server return codes
- List of HTTP header fields
- List of SMTP server return codes
- Common Log Format

References

1. "Hypertext Transfer Protocol (HTTP) Status Code Registry" (https://www.iana.org/assignments/http-status-codes/). Iana.org. Archived (https://web.archive.org/web/20111211100506/https://www.iana.org/assignments/http-status-codes) from the original on December 11, 2011. Retrieved January 8, 2015.
2. R. Fielding; M. Nottingham; J. Reschke, eds. (June 2022). *HTTP Semantics* (https://www.rfc-editor.org/rfc/rfc9110). Internet Engineering Task Force. doi:10.17487/RFC9110 (https://doi.org/10.17487%2FRFC9110). ISSN 2070-1721 (https://search.worldcat.org/issn/2070-1721). STD 97. RFC 9110 (https://datatracker.ietf.org/doc/html/rfc9110). *Internet Standard 97.* Obsoletes RFC 2818 (https://www.rfc-editor.org/rfc/rfc2818), 7230 (https://www.rfc-editor.org/rfc/rfc7230), 7231 (https://www.rfc-editor.org/rfc/rfc7231), 7232 (https://www.rfc-editor.org/rfc/rfc7232), 7233 (https://www.rfc-editor.org/rfc/rfc7233), 7235 (https://www.rfc-editor.org/rfc/rfc7235), 7538 (https://www.rfc-editor.org/rfc/rfc7538), 7615 (https://www.rfc-editor.org/rfc/rfc7615) and 7694 (https://www.rfc-editor.org/rfc/rfc7694). Updates RFC 3864 (https://www.rfc-editor.org/rfc/rfc3864).
3. Goland, Yaronn; Whitehead, Jim; Faizi, Asad; Carter, Steve R.; Jensen, Del (February 1999). *HTTP Extensions for Distributed Authoring – WEBDAV* (https://www.rfc-editor.org/rfc/rfc2518). Network Working Group. doi:10.17487/RFC2518 (https://doi.org/10.17487%2FRFC2518). RFC 2518 (https://datatracker.ietf.org/doc/html/rfc2518). *Proposed Standard.* Obsoleted by RFC 4918 (https://www.rfc-editor.org/rfc/rfc4918).

4. "102 Processing – HTTP MDN" (https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/102). July 25, 2023. 102 status code is deprecated
5. Oku, Kazuho (December 2017). *An HTTP Status Code for Indicating Hints* (https://www.rfc-editor.org/rfc/rfc8297). Internet Engineering Task Force. doi:10.17487/RFC8297 (https://doi.org/10.17487%2FRFC8297). RFC 8297 (https://datatracker.ietf.org/doc/html/rfc8297). *Experimental.*
6. Stewart, Mark; djna. "Create request with POST, which response codes 200 or 201 and content" (https://stackoverflow.com/questions/1860645/create-request-with-post-which-response-codes-200-or-201-and-content). *Stack Overflow*. Archived (https://web.archive.org/web/20161011010658/https://stackoverflow.com/questions/1860645/create-request-with-post-which-response-codes-200-or-201-and-content) from the original on October 11, 2016. Retrieved October 16, 2015.
7. Dusseault, Lisa, ed. (June 2007). *HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV)* (https://www.rfc-editor.org/rfc/rfc4918). Network Working Group. doi:10.17487/RFC4918 (https://doi.org/10.17487%2FRFC4918). RFC 4918 (https://datatracker.ietf.org/doc/html/rfc4918). *Proposed Standard.* Updated by RFC 5689 (https://www.rfc-editor.org/rfc/rfc5689). Obsoletes RFC 2518 (https://www.rfc-editor.org/rfc/rfc2518).
8. Hoff, Arthur van; Douglis, Fred; Krishnamurthy, Balachander; Goland, Yaron Y.; Hellerstein, Daniel M.; Feldmann, Anja; Mogul, Jeffrey (January 2002). *Delta encoding in HTTP* (https://www.rfc-editor.org/rfc/rfc3229). Network Working Group. doi:10.17487/RFC3229 (https://doi.org/10.17487%2FRFC3229). RFC 3229 (https://datatracker.ietf.org/doc/html/rfc3229). *Proposed Standard.*
9. Fielding; et al. (June 1999). *10.3.2 301 Moved Permanently* (https://www.rfc-editor.org/rfc/rfc2616#section-10.3.2). IETF. p. 61. sec. 10.3.2. doi:10.17487/RFC2616 (https://doi.org/10.17487%2FRFC2616). RFC 2616 (https://datatracker.ietf.org/doc/html/rfc2616).
10. "Site Move Tool" (https://www.bing.com/webmaster/help/how-to-use-the-site-move-tool-bb8f5112). *Bing Webmaster Help & How-to*.
11. "301 redirects" (https://support.google.com/webmasters/bin/answer.py?hl=en&answer=93633). *Google Webmaster Tools Help*.
12. T Berners-Lee; R. Fielding; H. Frystyk (May 1996). *Hypertext Transfer Protocol -- HTTP/1.0* (https://www.rfc-editor.org/rfc/rfc1945). Network Working Group. doi:10.17487/RFC1945 (https://doi.org/10.17487%2FRFC1945). RFC 1945 (https://datatracker.ietf.org/doc/html/rfc1945). *Informational.*
13. Lawrence, Eric. "HTTP Methods and Redirect Status Codes" (http://blogs.msdn.com/b/ieinternals/archive/2011/08/19/understanding-the-impact-of-redirect-response-status-codes-on-http-methods-like-head-get-post-and-delete.aspx). *EricLaw's IEInternals blog*. Retrieved August 20, 2011.
14. "Request and response objects | Django documentation | Django" (https://docs.djangoproject.com/en/dev/ref/request-response/#django.http.HttpResponseRedirect). Docs.djangoproject.com. Retrieved June 23, 2014.
15. Fielding, Roy T.; Reschke, Julian (June 2014). "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content" (https://tools.ietf.org/html/rfc7231#section-6.4.3). Tools.ietf.org. Retrieved January 5, 2019.
16. Bouquet, Paolo; Stoermer, Heiko; Vignolo, Massimiliano (January 12, 2011). "Web of Data and Web of Entities: Identity and Reference in Interlinked Data in the Semantic Web". *Philosophy & Technology*. **25**. Springer Nature: 5–26. doi:10.1007/s13347-010-0011-6 (https://doi.org/10.1007%2Fs13347-010-0011-6). ISSN 2210-5441 (https://search.worldcat.org/issn/2210-5441).
17. Halpin, Harry; Presutti, Valentina (2011). "The identity of resources on the Web: An ontology for Web architecture". *Applied Ontology*. **6** (3). IOS Press: 263–293. doi:10.3233/AO-2011-0095 (https://doi.org/10.3233%2FAO-2011-0095). ISSN 1875-8533 (https://search.worldcat.org/issn/1875-8533).

18. Allamaraju, Subbu; Allamaraju, Subrahmanyam (March 2010). *RESTful Web Services Cookbook*. O'Reilly Media. ISBN 9780596801687.
19. "Mozilla Bugzilla Bug 187996: Strange behavior on 305 redirect, comment 13" (https://web.archive.org/web/20140421051946/https://bugzilla.mozilla.org/show_bug.cgi?id=187996#c13). March 3, 2003. Archived from the original (https://bugzilla.mozilla.org/show_bug.cgi?id=187996#c13) on April 21, 2014. Retrieved May 21, 2009.
20. "The GNU Taler tutorial for PHP Web shop developers 0.4.0" (https://web.archive.org/web/20171108142249/https://docs.taler.net/merchant/frontend/php/html/tutorial.html#Headers-for-HTTP-402). *docs.taler.net*. Archived from the original (https://docs.taler.net/merchant/frontend/php/html/tutorial.html#Headers-for-HTTP-402) on November 8, 2017. Retrieved October 29, 2017.
21. "Google API Standard Error Responses" (https://web.archive.org/web/20170525005121/https://developers.google.com/doubleclick-search/v2/standard-error-responses#PAYMENT_REQUIRED). 2016. Archived from the original (https://developers.google.com/doubleclick-search/v2/standard-error-responses#PAYMENT_REQUIRED) on May 25, 2017. Retrieved June 21, 2017.
22. "Sipgate API Documentation" (https://api.sipgate.com/v2/doc/#/sessions/newCall). Archived (https://web.archive.org/web/20180710163601/https://api.sipgate.com/v2/doc#/sessions/newCall) from the original on July 10, 2018. Retrieved July 10, 2018.
23. "Shopify Documentation" (https://help.shopify.com/en/api/getting-started/response-status-codes). Archived (https://web.archive.org/web/20180725122914/https://help.shopify.com/en/api/getting-started/response-status-codes) from the original on July 25, 2018. Retrieved July 25, 2018.
24. "Stripe API Reference – Errors" (https://stripe.com/docs/api/errors). *stripe.com*. Retrieved October 28, 2019.
25. "HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV)" (https://web.archive.org/web/20160303200436/http://www.webdav.org/specs/rfc4918.html#rfc.section.9.1.1). IETF. June 2007. Archived from the original (http://www.webdav.org/specs/rfc4918.html#rfc.section.9.1.1) on March 3, 2016. Retrieved January 12, 2016.
26. "409 Conflict" (https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/409). *MDN Web Docs*. March 13, 2025. Retrieved June 11, 2025.
27. R. Fielding; J. Gettys; J. Mogul; H. Frystyk; L. Masinter; P. Leach; T. Berners-Lee (August 1999). *Hypertext Transfer Protocol -- HTTP/1.1* (https://www.rfc-editor.org/rfc/rfc2616). Network Working Group. doi:10.17487/RFC2616 (https://doi.org/10.17487%2FRFC2616). RFC 2616 (https://datatracker.ietf.org/doc/html/rfc2616). *Obsolete.* Obsoleted by RFC 7230 (https://www.rfc-editor.org/rfc/rfc7230), 7231 (https://www.rfc-editor.org/rfc/rfc7231), 7232 (https://www.rfc-editor.org/rfc/rfc7232), 7233 (https://www.rfc-editor.org/rfc/rfc7233), 7234 (https://www.rfc-editor.org/rfc/rfc7234) and 7235 (https://www.rfc-editor.org/rfc/rfc7235). Obsoletes RFC 2068 (https://www.rfc-editor.org/rfc/rfc2068). Updated by RFC 2817 (https://www.rfc-editor.org/rfc/rfc2817), 5785 (https://www.rfc-editor.org/rfc/rfc5785), 6266 (https://www.rfc-editor.org/rfc/rfc6266) and 6585 (https://www.rfc-editor.org/rfc/rfc6585).
28. TheDeadLike. "HTTP/1.1 Status Codes 400 and 417, cannot choose which" (http://serverfault.com/questions/433470/http-1-1-status-codes-400-and-417-cannot-choose-which). *serverFault*. Archived (https://web.archive.org/web/20151010125107/http://serverfault.com/questions/433470/http-1-1-status-codes-400-and-417-cannot-choose-which) from the original on October 10, 2015. Retrieved October 16, 2015.
29. L. Masinter (April 1, 1998). *Hyper Text Coffee Pot Control Protocol (HTCPCP/1.0)* (https://www.rfc-editor.org/rfc/rfc2324). Network Working Group. doi:10.17487/RFC2324 (https://doi.org/10.17487%2FRFC2324). RFC 2324 (https://datatracker.ietf.org/doc/html/rfc2324). *Informational.* Updated by RFC 7168 (https://www.rfc-editor.org/rfc/rfc7168). This is an April Fools' Day Request for Comments. "Any attempt to brew coffee with a teapot should result in the error code "418 I'm a teapot". The resulting entity body MAY be short and stout."

30. Barry Schwartz (August 26, 2014). "New Google Easter Egg For SEO Geeks: Server Status 418, I'm A Teapot" (https://web.archive.org/web/20151115041951/http://searchengineland.com/new-google-easter-egg-seo-geeks-server-status-418-im-teapot-201739). *Search Engine Land*. Archived from the original (http://searchengineland.com/new-google-easter-egg-seo-geeks-server-status-418-im-teapot-201739) on November 15, 2015. Retrieved November 4, 2015.
31. "Error 418 (I'm a teapot)!?" (https://www.google.com/teapot). Retrieved December 17, 2025.
32. "Enable extra web security on a website" (https://help.dreamhost.com/hc/en-us/articles/215947927-Enable-extra-web-security-on-a-website). *DreamHost*. Retrieved December 18, 2022.
33. "I Went to a Russian Website and All I Got Was This Lousy Teapot" (https://www.pcmag.com/news/i-went-to-a-russian-website-and-all-i-got-was-this-lousy-teapot). *PCMag*. February 25, 2022. Retrieved December 18, 2022.
34. M. Nottingham; R. Fielding (April 2012). *Additional HTTP Status Codes* (https://www.rfc-editor.org/rfc/rfc6585). Internet Engineering Task Force. doi:10.17487/RFC6585 (https://doi.org/10.17487%2FRFC6585). ISSN 2070-1721 (https://search.worldcat.org/issn/2070-1721). RFC 6585 (https://datatracker.ietf.org/doc/html/rfc6585). *Proposed Standard.* Updates RFC 2616 (https://www.rfc-editor.org/rfc/rfc2616).
35. Bray, T. (February 2016). "An HTTP Status Code to Report Legal Obstacles" (https://tools.ietf.org/html/rfc7725). *ietf.org*. Archived (https://web.archive.org/web/20160304040017/http://tools.ietf.org/html/rfc7725) from the original on March 4, 2016. Retrieved March 7, 2015.
36. Paul, Ian (December 21, 2015). "Error 451 is the new Ray Bradbury-inspired HTTP code for online censorship" (https://www.pcworld.com/article/418860/error-451-is-the-new-ray-bradbury-inspired-http-code-for-online-censorship.html). *PC World*. Retrieved July 18, 2025.
37. alex. "What is the correct HTTP status code to send when a site is down for maintenance?" (https://stackoverflow.com/questions/2786595/what-is-the-correct-http-status-code-to-send-when-a-site-is-down-for-maintenance). *Stack Overflow*. Archived (https://web.archive.org/web/20161011013125/https://stackoverflow.com/questions/2786595/what-is-the-correct-http-status-code-to-send-when-a-site-is-down-for-maintenance) from the original on October 11, 2016. Retrieved October 16, 2015.
38. K. Holtman; A.H. Mutz (March 1998). *Transparent Content Negotiation in HTTP* (https://www.rfc-editor.org/rfc/rfc2295). Network Working Group. doi:10.17487/RFC2295 (https://doi.org/10.17487%2FRFC2295). RFC 2295 (https://datatracker.ietf.org/doc/html/rfc2295). *Experimental.*
39. Nielsen, Henrik Frystyk; Leach, Paul; Lawrence, Scott (February 2000). *An HTTP Extension Framework* (https://www.rfc-editor.org/rfc/rfc2774). Network Working Group. doi:10.17487/RFC2774 (https://doi.org/10.17487%2FRFC2774). RFC 2774 (https://datatracker.ietf.org/doc/html/rfc2774). *Historic.*
40. "The HTTP status codes in IIS 7.0" (https://support.microsoft.com/kb/943891/). Microsoft. July 14, 2009. Archived (https://web.archive.org/web/20090409091609/http://support.microsoft.com/kb/943891) from the original on April 9, 2009. Retrieved April 1, 2009.
41. "Error message when you try to log on to Exchange 2007 by using Outlook Web Access: "440 Login Time-out" " (https://support.microsoft.com/en-us/help/941201/). Microsoft. 2010. Retrieved November 13, 2013.
42. "2.2.6 449 Retry With Status Code" (http://msdn.microsoft.com/en-us/library/dd891478(PROT.10).aspx). Microsoft. 2009. Archived (https://web.archive.org/web/20091005090136/http://msdn.microsoft.com/en-us/library/dd891478(PROT.10).aspx) from the original on October 5, 2009. Retrieved October 26, 2009.

43. "Screenshot of error page" (https://web.archive.org/web/20130511045107/https://public.bn1.livefilestore.com/y1pJXeg_sNOONKwMraE-xmZFWAfZF6COAKnyfgc-2ykUof743pV4XuRqm14pj-b_yK8Km4sfSR6mU5OhLrupZ8dFg). Archived from the original (https://public.bn1.livefilestore.com/y1pJXeg_sNOONKwMraE-xmZFWAfZF6COAKnyfgc-2ykUof743pV4XuRqm14pj-b_yK8Km4sfSR6mU5OhLrupZ8dFg) (bmp) on May 11, 2013. Retrieved October 11, 2009.
44. "MS-ASCMD, Section 3.1.5.2.2" (https://msdn.microsoft.com/en-us/library/gg651019). Msdn.microsoft.com. Archived (https://web.archive.org/web/20150326120838/https://msdn.microsoft.com/en-us/library/gg651019) from the original on March 26, 2015. Retrieved January 8, 2015.
45. "Ms-oxdisco" (http://msdn.microsoft.com/en-us/library/cc433481). Msdn.microsoft.com. Archived (https://web.archive.org/web/20140731074358/http://msdn.microsoft.com/en-us/library/cc433481) from the original on July 31, 2014. Retrieved January 8, 2015.
46. "ngx_http_request.h" (https://web.archive.org/web/20170919111558/http://lxr.nginx.org/source/src/http/ngx_http_request.h). *nginx 1.9.5 source code*. nginx inc. Archived from the original (http://lxr.nginx.org/source/src/http/ngx_http_request.h) on September 19, 2017. Retrieved January 9, 2016.
47. "ngx_http_special_response.c" (https://web.archive.org/web/20180508052017/http://lxr.nginx.org/source/src/http/ngx_http_special_response.c). *nginx 1.9.5 source code*. nginx inc. Archived from the original (http://lxr.nginx.org/source/src/http/ngx_http_special_response.c) on May 8, 2018. Retrieved January 9, 2016.
48. "return" directive (http://nginx.org/en/docs/http/ngx_http_rewrite_module.html#return) Archived (https://web.archive.org/web/20180301164309/http://nginx.org/en/docs/http/ngx_http_rewrite_module.html#return) March 1, 2018, at the Wayback Machine (http_rewrite module) documentation.
49. "Troubleshooting: Error Pages" (https://web.archive.org/web/20160304035212/https://support.cloudflare.com/hc/en-us/sections/200820298-Error-Pages). Cloudflare. Archived from the original (https://support.cloudflare.com/hc/en-us/articles/115003014432-HTTP-Status-Codes) on March 4, 2016. Retrieved January 9, 2016.
50. "Error 520: web server returns an unknown error" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-520/). Cloudflare. May 29, 2025.
51. "Error 521: web server is down" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-521/). Cloudflare. April 29, 2025.
52. "Error 522: connection timed out" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-522/). Cloudflare. May 5, 2025.
53. "Error 523: origin is unreachable" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-523/). Cloudflare. April 29, 2025.
54. "Error 524: a timeout occurred" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-524/). Cloudflare. August 6, 2025.
55. "Error 525: SSL handshake failed" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-525/). Cloudflare. April 29, 2025.
56. "Cloudflare support docs" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-525/). *developers.cloudflare.com*. April 29, 2025. Retrieved September 14, 2025.
57. "Error 526: invalid SSL certificate" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-526/). Cloudflare. August 11, 2025.
58. "Cloudflare support docs" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-526/). *developers.cloudflare.com*. August 11, 2025. Retrieved September 14, 2025.

59. "527 Error: Railgun Listener to origin error" (https://web.archive.org/web/20161013152120/https://support.cloudflare.com/hc/en-us/articles/217891268-527-Railgun-Listener-to-Origin-Error). Cloudflare. Archived from the original (https://developers.cloudflare.com/support/troubleshooting/cloudflare-errors/troubleshooting-cloudflare-5xx-errors/#527-error-railgun-listener-to-origin-error) on October 13, 2016. Retrieved October 12, 2016.
60. "Error 530" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-530/). Cloudflare. Retrieved November 1, 2019.
61. "Cloudflare support docs" (https://developers.cloudflare.com/support/troubleshooting/http-status-codes/cloudflare-5xx-errors/error-530/). *developers.cloudflare.com*. April 29, 2025. Retrieved September 14, 2025.
62. "Troubleshoot Your Application Load Balancers – Elastic Load Balancing" (https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-troubleshooting.html). *docs.aws.amazon.com*. Retrieved May 17, 2023.
63. "HTTP Error Codes and Quick Fixes" (https://web.archive.org/web/20151123100640/https://documentation.cpanel.net/display/CKB/HTTP+Error+Codes+and+Quick+Fixes#HTTPErrorCodesandQuickFixes-509BandwidthLimitExceeded). Docs.cpanel.net. Archived from the original (https://documentation.cpanel.net/display/CKB/HTTP+Error+Codes+and+Quick+Fixes#HTTPErrorCodesandQuickFixes-509BandwidthLimitExceeded) on November 23, 2015. Retrieved October 15, 2015.
64. "framework/src/Illuminate/Foundation/Exceptions/Handler.php" (https://github.com/laravel/framework/blob/57ae89a8acbc316153315272a4af2e68370f5e36/src/Illuminate/Foundation/Exceptions/Handler.php#L492). *GitHub*. Retrieved December 12, 2023.
65. "draft-ietf-webdav-protocol-05: Extensions for Distributed Authoring on the World Wide Web -- WEBDAV" (https://datatracker.ietf.org/doc/html/draft-ietf-webdav-protocol-05#section-10.5).
66. "Enum HttpStatus" (https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/http/HttpStatus.html). *Spring Framework*. org.springframework.http. Archived (https://web.archive.org/web/20151025102238/http://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/http/HttpStatus.html) from the original on October 25, 2015. Retrieved October 16, 2015.
67. "Twitter Error Codes & Responses" (https://web.archive.org/web/20170927155109/https://developer.twitter.com/en/docs/basics/response-codes). Twitter. 2014. Archived from the original (https://developer.twitter.com/en/docs/basics/response-codes) on September 27, 2017. Retrieved January 20, 2014.
68. "HTTP Status Codes and SEO: what you need to know" (https://www.contentkingapp.com/academy/http-status-codes/). *ContentKing*. Retrieved August 9, 2019.
69. "Shopify API response status and error codes" (https://shopify.dev/docs/api/usage/response-codes). Retrieved December 12, 2023.
70. "Using token-based authentication" (https://web.archive.org/web/20140926033953/http://help.arcgis.com/en/arcgisserver/10.0/apis/soap/index.htm#Using_token_authentication.htm). *ArcGIS Server SOAP SDK*. Archived from the original (http://help.arcgis.com/en/arcgisserver/10.0/apis/soap/index.htm#Using_token_authentication.htm) on September 26, 2014. Retrieved September 8, 2014.
71. "Receiving a "508 Resource Limit Is Reached" error when browsing a site on CloudLinux" (https://web.archive.org/web/20250719083333/https://support.cpanel.net/hc/en-us/articles/360057457313-Receiving-a-508-Resource-Limit-Is-Reached-error-when-browsing-a-site-on-CloudLinux). September 28, 2021. Archived from the original (https://support.cpanel.net/hc/en-us/articles/360057457313-Receiving-a-508-Resource-Limit-Is-Reached-error-when-browsing-a-site-on-CloudLinux) on July 19, 2025. Retrieved July 19, 2025.
72. "SSL Labs API v3 Documentation" (https://github.com/ssllabs/ssllabs-scan/blob/master/ssllabs-api-docs-v3.md#error-response-status-codes). *github.com*.

73. "Platform Considerations | Pantheon Docs" (https://web.archive.org/web/20170106011710/https://pantheon.io/docs/platform-considerations/). *pantheon.io*. Archived from the original (https://pantheon.io/docs/platform-considerations/) on January 6, 2017. Retrieved January 5, 2017.
74. Seen in 2017 (https://learn.microsoft.com/en-us/archive/msdn-technet-forums/5a4f8eb5-bf1b-4776-b4bb-4baef621838f), 2024[link removed] and 2025 examples (https://github.com/sherlock-project/sherlock/issues/2652)
75. "218 This is fine – HTTP status code explained" (https://http.dev/218). *HTTP.dev*. Retrieved July 25, 2023.
76. "HTTP status codes – ascii-code.com" (https://web.archive.org/web/20170107115008/http://www.ascii-code.com/http-status-codes.php). *www.ascii-code.com*. Archived from the original (http://www.ascii-code.com/http-status-codes.php) on January 7, 2017. Retrieved December 23, 2016.

External links

- Hypertext Transfer Protocol (HTTP) Status Code Registry (https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml) at the *Internet Assigned Numbers Authority*
- MDN status code reference (https://developer.mozilla.org/en-US/docs/Web/HTTP/Status) at *mozilla.org*